

HTTP请求示例

最近更新时间：2025.03.05 11:25:02

首次发布时间：2024.11.28 11:59:52

复制全文 我的收藏



前置说明（必读）

以下内容为直接使用HTTP方式调用接口的请求示例(包含签名过程)，替换关键信息即可直接调用。

Step1(必需): 通过火山访问控制获取AK/SK，需确保火山账号已开通对应权限和相关策略

正确替换示例中的 `AccessKeyID` 和 `SecretAccessKey` 参数值

无权限情况下会报错“”

Step2(大多数情况下可跳过): 查看接口文档 `请求参数-Query参数` 中的 `Action` 及对应 `Version`

正确替换示例中的 `action` 和 `version` 参数值

Step3(必需): 查看接口文档 `请求参数-Body参数、请求示例`，将 `请求示例` 内容复制到 `调用示例的body`入参部分

正确替换示例中的Body入参

Step4: 运行程序进行调试即可，待测试调通后，可以按需集成到项目中(不建议直接Copy，建议自行适配或更新包)

调用示例

Java

```
package com.volcengine.example.cv;

import com.alibaba.fastjson.JSONObject;
import com.google.common.io.ByteStreams;
import com.volcengine.helper.Utills;
import org.apache.commons.codec.binary.Hex;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.ByteBuffer;
```

```
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.text.SimpleDateFormat;
import java.util.*;

/**
 * Copyright (year) Beijing Volcano Engine Technology Ltd.
 * <p>
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 * <p>
 * http://www.apache.org/licenses/LICENSE-2.0
 * <p>
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
public class Sign {
    private static final BitSet URLENCODER = new BitSet(256);
    private static final String CONST_ENCODE = "0123456789ABCDEF";
    private static final Charset UTF_8 = StandardCharsets.UTF_8;
    private final String region;
    private final String service;
    private final String schema;
    private final String host;
    private final String path;
    private final String ak;
    private final String sk;

    static {
        int i;
        for (i = 97; i <= 122; ++i) {
            URLENCODER.set(i);
        }
        for (i = 65; i <= 90; ++i) {
            URLENCODER.set(i);
        }
        for (i = 48; i <= 57; ++i) {
            URLENCODER.set(i);
        }
    }
}
```

```
        URLEncoder.set('-');
        URLEncoder.set('_');
        URLEncoder.set('.');
        URLEncoder.set('~');
    }

    public static void main(String[] args) throws Exception {
        // 火山官网密钥信息, 注意sk结尾有==
        String AccessKeyID = "AK*****";
        String SecretAccessKey = "*****==";
        // 请求域名
        String endpoint = "visual.volcengineapi.com";
        String path = "/"; // 路径, 不包含 Query// 请求接口信息
        String service = "cv";
        String region = "cn-north-1";
        String schema = "https";
        Sign sign = new Sign(region, service, schema, endpoint, path, AccessK
        // 参考接口文档Query参数
        String action = "CVProcess";
        String version = "2022-08-31";
        Date date = new Date();
        // 参考接口文档Body参数
        JSONObject req=new JSONObject();
        req.put("req_key","xxx");
        ArrayList<String> imageUrls = new ArrayList<String>();
        imageUrls.add("*****");
        req.put("image_urls",imageUrls);
        req.put("prompt","*****");

        sign.doRequest("POST", new HashMap(), req.toJSONString().getBytes(),
    }

    public Sign(String region, String service, String schema, String host, St
        this.region = region;
        this.service = service;
        this.host = host;
        this.schema = schema;
        this.path = path;
        this.ak = ak;
        this.sk = sk;
    }

    public void doRequest(String method, Map<String, String> queryList, byte[]
        Date date, String action, String version) throws Ex
```

```

    if (body == null) {
        body = new byte[0];
    }
    String xContentSha256 = hashSHA256(body);
    SimpleDateFormat sdf = new SimpleDateFormat("yyyyMMdd'T'HHmmss'Z'");
    sdf.setTimeZone(TimeZone.getTimeZone("GMT"));
    String xDate = sdf.format(date);
    String shortXDate = xDate.substring(0, 8);
    String contentType = "application/json";
    String signHeader = "host;x-date;x-content-sha256;content-type";

    SortedMap<String, String> realQueryList = new TreeMap<>(queryList);
    realQueryList.put("Action", action);
    realQueryList.put("Version", version);
    StringBuilder querySB = new StringBuilder();
    for (String key : realQueryList.keySet()) {
        querySB.append(signStringEncoder(key)).append("=").append(signStr
    }
    querySB.deleteCharAt(querySB.length() - 1);

    String canonicalStringBuilder = method + "\n" + path + "\n" + querySB
        "host:" + host + "\n" +
        "x-date:" + xDate + "\n" +
        "x-content-sha256:" + xContentSha256 + "\n" +
        "content-type:" + contentType + "\n" +
        "\n" +
        signHeader + "\n" +
        xContentSha256;

    System.out.println(canonicalStringBuilder);

    String hashcanonicalString = hashSHA256(canonicalStringBuilder.getBytes());
    String credentialScope = shortXDate + "/" + region + "/" + service +
    String signString = "HMAC-SHA256" + "\n" + xDate + "\n" + credentialS

    byte[] signKey = genSigningSecretKeyV4(sk, shortXDate, region, servic
//    String signature = HexFormat.of().formatHex(hmacSHA256(signKey, sig
    String signature = Hex.encodeHexString(Utils.hmacSHA256(signKey, sign

    URL url = new URL(schema + "://" + host + path + "?" + querySB);

    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod(method);
    conn.setRequestProperty("Host", host);

```

```
conn.setRequestProperty("X-Date", xDate);
conn.setRequestProperty("X-Content-Sha256", xContentSha256);
conn.setRequestProperty("Content-Type", contentType);
conn.setRequestProperty("Authorization", "HMAC-SHA256" +
    " Credential=" + ak + "/" + credentialScope +
    ", SignedHeaders=" + signHeader +
    ", Signature=" + signature);
if (!Objects.equals(conn.getRequestMethod(), "GET")) {
    conn.setDoOutput(true);
    OutputStream os = conn.getOutputStream();
    os.write(body);
    os.flush();
    os.close();
}
conn.connect();

int responseCode = conn.getResponseCode();

InputStream is;
if (responseCode == 200) {
    is = conn.getInputStream();
} else {
    is = conn.getErrorStream();
}
String responseBody = new String(ByteStreams.toByteArray(is));
is.close();

System.out.println(responseCode);
System.out.println(responseBody);
}

private String signStringEncoder(String source) {
    if (source == null) {
        return null;
    }
    StringBuilder buf = new StringBuilder(source.length());
    ByteBuffer bb = UTF_8.encode(source);
    while (bb.hasRemaining()) {
        int b = bb.get() & 255;
        if (URLENCODER.get(b)) {
            buf.append((char) b);
        } else if (b == 32) {
            buf.append("%20");
        } else {

```

```
        buf.append("%");
        char hex1 = CONST_ENCODE.charAt(b >> 4);
        char hex2 = CONST_ENCODE.charAt(b & 15);
        buf.append(hex1);
        buf.append(hex2);
    }
}

return buf.toString();
}

public static String hashSHA256(byte[] content) throws Exception {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");

        return Hex.encodeHexString(md.digest(content));
    } catch (Exception e) {
        throw new Exception(
            "Unable to compute hash while signing request: "
            + e.getMessage(), e);
    }
}

public static byte[] hmacSHA256(byte[] key, String content) throws Except
    try {
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(key, "HmacSHA256"));
        return mac.doFinal(content.getBytes());
    } catch (Exception e) {
        throw new Exception(
            "Unable to calculate a request signature: "
            + e.getMessage(), e);
    }
}

private byte[] genSigningSecretKeyV4(String secretKey, String date, Strin
    byte[] kDate = hmacSHA256((secretKey).getBytes(), date);
    byte[] kRegion = hmacSHA256(kDate, region);
    byte[] kService = hmacSHA256(kRegion, service);
    return hmacSHA256(kService, "request");
}
}
```

Golang

```
/*
Copyright (year) Beijing Volcano Engine Technology Ltd.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package main

import (
    "bytes"
    "crypto/hmac"
    "crypto/sha256"
    "encoding/hex"
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "net/http/httputil"
    "net/url"
    "strings"
    "time"
)

const (
    // 请求凭证, 从访问控制申请
    AccessKeyID      = "AK*****"
    SecretAccessKey = "*****=="

    // 请求地址
    Addr = "https://visual.volcengineapi.com"
    Path = "/" // 路径, 不包含 Query
```

```
// 请求接口信息
Service = "cv"
Region  = "cn-north-1"

// 请求Query信息
Action  = "CVProcess"
Version = "2022-08-31"
)

func hmacSHA256(key []byte, content string) []byte {
    mac := hmac.New(sha256.New, key)
    mac.Write([]byte(content))
    return mac.Sum(nil)
}

func getSignedKey(secretKey, date, region, service string) []byte {
    kDate := hmacSHA256([]byte(secretKey), date)
    kRegion := hmacSHA256(kDate, region)
    kService := hmacSHA256(kRegion, service)
    kSigning := hmacSHA256(kService, "request")

    return kSigning
}

func hashSHA256(data []byte) []byte {
    hash := sha256.New()
    if _, err := hash.Write(data); err != nil {
        log.Printf("input hash err:%s", err.Error())
    }

    return hash.Sum(nil)
}

func doRequest(method string, queries url.Values, body []byte) ([]byte, int,
// 1. 构建请求
queries.Set("Action", Action)
queries.Set("Version", Version)
requestAddr := fmt.Sprintf("%s%s?%s", Addr, Path, queries.Encode())
log.Printf("request addr: %s\n", requestAddr)

request, err := http.NewRequest(method, requestAddr, bytes.NewBuffer(
if err != nil {
    return nil, 0, fmt.Errorf("bad request: %w", err)
}
```



```

// 2. 构建签名材料
now := time.Now()
date := now.UTC().Format("20060102T150405Z")
authDate := date[:8]
request.Header.Set("X-Date", date)

payload := hex.EncodeToString(hashSHA256(body))
request.Header.Set("X-Content-Sha256", payload)
request.Header.Set("Content-Type", "application/json")

queryString := strings.Replace(queries.Encode(), "+", "%20", -1)
signedHeaders := []string{"host", "x-date", "x-content-sha256", "cont
var headerList []string
for _, header := range signedHeaders {
    if header == "host" {
        headerList = append(headerList, header+": "+request.Ho
    } else {
        v := request.Header.Get(header)
        headerList = append(headerList, header+": "+strings.Tr
    }
}
headerString := strings.Join(headerList, "\n")

canonicalString := strings.Join([]string{
    method,
    Path,
    queryString,
    headerString + "\n",
    strings.Join(signedHeaders, ";"),
    payload,
}, "\n")
log.Printf("canonical string:\n%s\n", canonicalString)

hashedCanonicalString := hex.EncodeToString(hashSHA256([]byte(canonic
log.Printf("hashed canonical string: %s\n", hashedCanonicalString)

credentialScope := authDate + "/" + Region + "/" + Service + "/reques
signString := strings.Join([]string{
    "HMAC-SHA256",
    date,
    credentialScope,
    hashedCanonicalString,
}, "\n")

```

```
log.Printf("sign string:\n%s\n", signString)

// 3. 构建认证请求头
signedKey := getSignedKey(SecretAccessKey, authDate, Region, Service)
signature := hex.EncodeToString(hmacSHA256(signedKey, signString))
log.Printf("signature: %s\n", signature)

authorization := "HMAC-SHA256" +
    " Credential=" + AccessKeyID + "/" + credentialScope +
    ", SignedHeaders=" + strings.Join(signedHeaders, ";") +
    ", Signature=" + signature

request.Header.Set("Authorization", authorization)
log.Printf("authorization: %s\n", authorization)

// 4. 打印请求, 发起请求
requestRaw, err := httputil.DumpRequest(request, true)
if err != nil {
    return nil, 0, fmt.Errorf("dump request err: %w", err)
}

log.Printf("request:\n%s\n", string(requestRaw))

response, err := http.DefaultClient.Do(request)
if err != nil {
    return nil, 0, fmt.Errorf("do request err: %w", err)
}

// 5. 打印响应
responseRaw, err := httputil.DumpResponse(response, true)
if err != nil {
    return nil, 0, fmt.Errorf("dump response err: %w", err)
}
respStr := strings.ReplaceAll(string(responseRaw), "\\u0026", "&")
log.Printf("response:\n%s\n", respStr)
return requestRaw, response.StatusCode, err
}

func main() {
    // 请求Body信息, 参考接口文档请求示例
    reqBody := map[string]interface{}{
        "req_key": "*****",
        //.....
    }
}
```

```
reqBodyStr, _ := json.Marshal(reqBody)
_, _, _ = doRequest("POST", url.Values{}, reqBodyStr)
}
```

Python

```
import json
import sys
import os
import base64
import datetime
import hashlib
import hmac
import requests

method = 'POST'
host = 'visual.volcengineapi.com'
region = 'cn-north-1'
endpoint = 'https://visual.volcengineapi.com'
service = 'cv'

def sign(key, msg):
    return hmac.new(key, msg.encode('utf-8'), hashlib.sha256).digest()

def getSignatureKey(key, dateStamp, regionName, serviceName):
    kDate = sign(key.encode('utf-8'), dateStamp)
    kRegion = sign(kDate, regionName)
    kService = sign(kRegion, serviceName)
    kSigning = sign(kService, 'request')
    return kSigning

def formatQuery(parameters):
    request_parameters_init = ''
    for key in sorted(parameters):
        request_parameters_init += key + '=' + parameters[key] + '&'
    request_parameters = request_parameters_init[:-1]
    return request_parameters

def signV4Request(access_key, secret_key, service, req_query, req_body):
    if access_key is None or secret_key is None:
        print('No access key is available.')
```

```

sys.exit()

t = datetime.datetime.utcnow()
current_date = t.strftime('%Y%m%dT%H%M%SZ')
# current_date = '20210818T095729Z'
datestamp = t.strftime('%Y%m%d') # Date w/o time, used in credential scope
canonical_uri = '/'
canonical_querystring = req_query
signed_headers = 'content-type;host;x-content-sha256;x-date'
payload_hash = hashlib.sha256(req_body.encode('utf-8')).hexdigest()
content_type = 'application/json'
canonical_headers = 'content-type:' + content_type + '\n' + 'host:' + host + \
    '\n' + 'x-content-sha256:' + payload_hash + \
    '\n' + 'x-date:' + current_date + '\n'
canonical_request = method + '\n' + canonical_uri + '\n' + canonical_querystring + \
    '\n' + canonical_headers + '\n' + signed_headers + '\n' + payload_hash
# print(canonical_request)
algorithm = 'HMAC-SHA256'
credential_scope = datestamp + '/' + region + '/' + service + '/' + 'request'
string_to_sign = algorithm + '\n' + current_date + '\n' + credential_scope + \
    canonical_request.encode('utf-8')).hexdigest()
# print(string_to_sign)
signing_key = getSignatureKey(secret_key, datestamp, region, service)
# print(signing_key)
signature = hmac.new(signing_key, (string_to_sign).encode('utf-8'), hashlib.sha256).hexdigest()
# print(signature)

authorization_header = algorithm + ' ' + 'Credential=' + access_key + '/' + \
    credential_scope + ', ' + 'SignedHeaders=' + \
    signed_headers + ', ' + 'Signature=' + signature
# print(authorization_header)
headers = {'X-Date': current_date,
           'Authorization': authorization_header,
           'X-Content-Sha256': payload_hash,
           'Content-Type': content_type
          }
# print(headers)

# ***** SEND THE REQUEST *****
request_url = endpoint + '?' + canonical_querystring

print('\nBEGIN REQUEST++++++++++++++++++++++++++++++++++++')
print('Request URL = ' + request_url)

```

```

try:
    r = requests.post(request_url, headers=headers, data=req_body)
except Exception as err:
    print(f'error occurred: {err}')
    raise
else:
    print('\nRESPONSE+++++')
    print(f'Response code: {r.status_code}\n')
    # 使用 replace 方法将 \u0026 替换为 &
    resp_str = r.text.replace("\u0026", "&")
    print(f'Response body: {resp_str}\n')

if __name__ == "__main__":
    # 请求凭证, 从访问控制申请
    access_key = 'AK*****'
    secret_key = '*****=='

    # 请求Query, 按照接口文档中填入即可
    query_params = {
        'Action': 'CVProcess',
        'Version': '2022-08-31',
    }
    formatted_query = formatQuery(query_params)

    # 请求Body, 按照接口文档中填入即可
    body_params = {
        "req_key": "*****",
        # .....
    }
    formatted_body = json.dumps(body_params)

    signV4Request(access_key, secret_key, service,
                   formatted_query, formatted_body)

```

PHP

```

<?php
/**
 * Copyright (year) Beijing Volcano Engine Technology Ltd.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");

```

```
* you may not use this file except in compliance with the License.
* You may obtain a copy of the License at
*
* http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing, software
* distributed under the License is distributed on an "AS IS" BASIS,
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
* See the License for the specific language governing permissions and
* limitations under the License.
*/
require('../vendor/autoload.php');
// 需要自行安装 composer (https://getcomposer.org/doc/00-intro.md), 并安装GuzzleH
use GuzzleHttp\Client;
use GuzzleHttp\Exception\GuzzleException;

// 基础信息, 基本不用变更
$Host = "visual.volcengineapi.com";
$ContentType = "application/json";
$Service = "cv";
$Region = "cn-north-1";

/**
 * @throws GuzzleException
 */
// 第一步: 创建一个 API 请求函数。签名计算的过程包含在该函数中。
function request($method, $query, $header, $ak, $sk, $action, $version, $body)
{
    // 第二步: 创建身份证明。其中的 Service 和 Region 字段是固定的。ak 和 sk 分别代表
    // AccessKeyId 和 SecretAccessKey。同时需要初始化签名结构体。一些签名计算时需要的,
    // 初始化身份证明结构体
    global $Service, $Region, $Host, $ContentType;
    $credential = [
        'accessKeyId' => $ak,
        'secretKeyId' => $sk,
        'service' => $Service,
        'region' => $Region,
    ];
    // 初始化签名结构体
    $query = array_merge($query, [
        'Action' => $action,
        'Version' => $version
    ]);
    ksort($query);
```

```

$requestParam = [
    // body是http请求需要的原生body
    'body' => $body,
    'host' => $Host,
    'path' => '/',
    'method' => $method,
    'contentType' => $ContentType,
    'date' => gmdate('Ymd\THis\Z'),
    'query' => $query
];

// 第三步: 接下来开始计算签名。在计算签名前, 先准备好用于接收签算结果的 signResult 变
// 初始化签名结果的结构体
$xDate = $requestParam['date'];
$shortXDate = substr($xDate, 0, 8);
$xContentSha256 = hash('sha256', $requestParam['body']);
$signResult = [
    'Host' => $requestParam['host'],
    'X-Content-Sha256' => $xContentSha256,
    'X-Date' => $xDate,
    'Content-Type' => $requestParam['contentType']
];

// 第四步: 计算 Signature 签名。
$signedHeaderStr = join(';', ['content-type', 'host', 'x-content-sha256',
$canonicalRequestStr = join("\n", [
    $requestParam['method'],
    $requestParam['path'],
    http_build_query($requestParam['query']),
    join("\n", ['content-type:'. $requestParam['contentType'], 'host:'. $
    '',
    $signedHeaderStr,
    $xContentSha256
]);
$hashedCanonicalRequest = hash("sha256", $canonicalRequestStr);
$credentialScope = join('/', [$shortXDate, $credential['region'], $ creden
$stringToSign = join("\n", ['HMAC-SHA256', $xDate, $credentialScope, $has
$kDate = hash_hmac("sha256", $shortXDate, $credential['secretKeyId'], tru
$kRegion = hash_hmac("sha256", $credential['region'], $kDate, true);
$kService = hash_hmac("sha256", $credential['service'], $kRegion, true);
$kSigning = hash_hmac("sha256", 'request', $kService, true);
$signature = hash_hmac("sha256", $stringToSign, $kSigning);
$signResult['Authorization'] = sprintf("HMAC-SHA256 Credential=%s, Signed
$header = array_merge($header, $signResult);
// 第五步: 将 Signature 签名写入 HTTP Header 中, 并发送 HTTP 请求。
$client = new Client([

```

```
'base_uri' => 'https://'. $requestParam['host'],
'timeout' => 120.0,
]);
$response = $client->request($method, 'https://'. $requestParam['host'].
    'headers' => $header,
    'query' => $requestParam['query'],
    'body' => $requestParam['body']
]);
$responseContent = $response->getBody()->getContents();
// 转换 \u0026 为 &
$responseContent = str_replace('\u0026', '&', $responseContent);
return $responseContent;
}
$now = time();
// 火山官网密钥信息, 注意sk结尾有==
$AccessKeyID = 'AK*****';
$SecretAccessKey = '*****==';

// 参考接口文档Query参数
$action = "CVProcess";
$version = "2022-08-31";

// 参考接口文档Body参数
$requestBody = [
    "req_key" => "*****",
    //.....
];
$body = json_encode($requestBody);

try {
    $response = request("POST", [], [], $AccessKeyID, $SecretAccessKey, $action);
    print_r($response);
} catch (GuzzleException $e) {
    print_r($e->getResponse()->getBody()->getContents() . "\n");
}
```